

1. Non-gated MoE Activation Fusion in QuACK / Sonic-MoE integration

1. Summary

Sonic-MoE의 고성능 MoE 커널에는 SwiGLU와 같은 gate 계열 activation function을 빠르게 처리하는 backward 최적화가 포함되어 있었습니다. 반면 `squared_relu`와 같은 non-gated activation function은 같은 최적화 경로가 구현되어 있지 않아, activation gradient 계산, token별 router score 곱셈, router-score gradient 계산을 별도의 PyTorch operation으로 처리해야 했습니다.

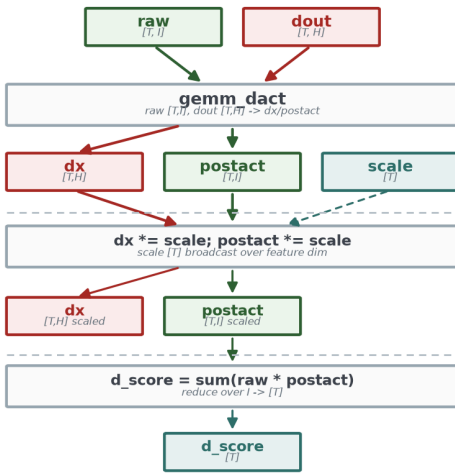
이러한 병목을 줄이기 위해 non-gated activation도 기존의 빠른 backward 경로 안에서 함께 처리하도록 커널 내부 연산을 확장했습니다. 이를 통해 dgrad backward는 `torch.compile` baseline 대비 최대 1.25배, Sonic-MoE end-to-end MoE kernel은 1.08배 속도 상승을 관찰했습니다.

2. Analysis

BACKWARD

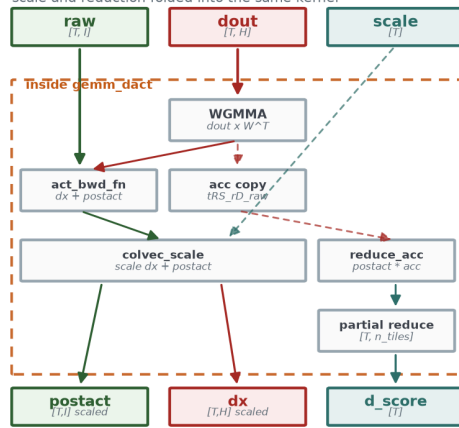
Non-gated expert MLP activation backward inside MoE

BEFORE



AFTER

scale and reduction folded into the same kernel



activation backward가 끝난 뒤 token별 router score를 곱하고 router의 gradient를 계산하면, dx와 activation 중간 결과를 HBM에 저장한 뒤 다시 읽어야 합니다. 이 후처리는 memory-bound operation이기 때문에, 작은 PyTorch operation으로 분리될수록 kernel launch overhead와 HBM IO의 비중이 큼니다.

게다가 activation backward kernel 내부에서는 이미 token별 raw, dout, activation 중간 결과를 tile 단위로 들고 있습니다. token별 router score 곱셈은 각 token row에 같은 scale을 broadcast해 곱하는 단순 elementwise 연산이므로, 결과를 HBM에 쓰기 전에 kernel에 통합하기가 적합합니다. 또한 register pressure도 낮음을 확인했기 때문에 gemm 이후 epilogue에 fusion 하기에 적절하다고 판단했습니다.

3. Implementation

- QuACK `gemm_dact`에 non-gated activation용 `colvec_scale` support 추가
- `colvec_scale`로 dx / dh와 recomputed activation에 per-token scale 적용
- `colvec_reduce`로 router-score gradient에 필요한 `sum(raw * act(h))` reduction 누적
- regular / varlen-M `gemm_dact` 결과를 PyTorch float32 reference와 비교하는 correctness test 추가
- Sonic-MoE non-gated backward epilogue fusion 적용을 위한 QuACK-side primitive 확장

4. Results

Sonic-MoE가 사용하는 라이브러리 QuACK 에 PR #143로 non-gated `gemm_dact` `colvec` 지원 기능을 구현 및 제출했습니다. Sonic-MoE에서 `squared_relu` 등의 non-gated activation family를 fused backward path로 지원하기 위한 기반 작업입니다. 이를 통해 dgrad backward path는 `torch.compile` baseline 대비 최대 1.25x speedup을 확인했으며 Sonic-MoE end-to-end MoE kernel은 1.08배 speedup을 확인했습니다..

2. Fused Linear Cross Entropy in Megatron-LM

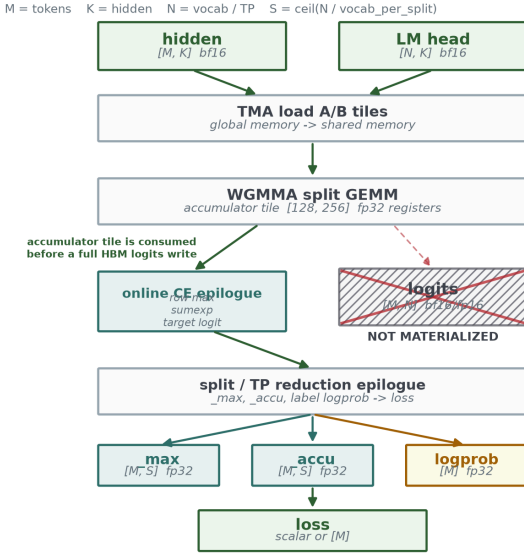
1. Summary

Megatron-LM에 Hopper GPU용 fused linear cross entropy kernel을 구현·제출했습니다. 기존 fused linear cross entropy 구현은 Blackwell GPU를 대상으로 하고 있어, 실제 사용 중인 Hopper 환경에서는 적용할 수 없었습니다. LigerKernel과 같은 대안도 있었지만, CuTe DSL/CUTLASS를 학습하고 Megatron-LM training path에 통합 가능한 형태로 구현하기 위해 직접 kernel을 작성했습니다.

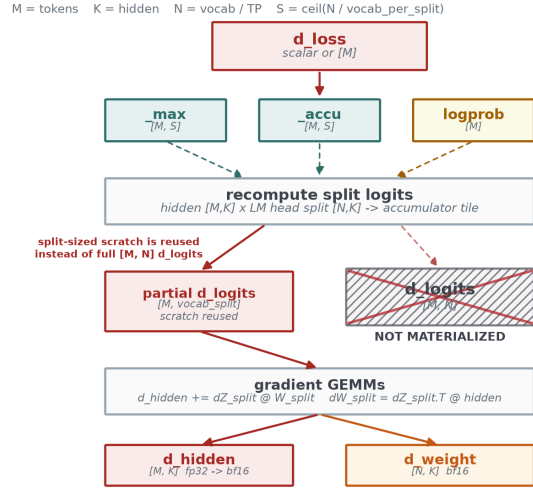
기존 구현은 lm head의 output인 logits [tokens, vocab/tp]을 HBM에 저장한 뒤 cross entropy와 backward 연산에서 다시 읽기 때문에, vocabulary size가 큰 LLM에서는 activation memory를 크게 차지합니다. 이 작업에서는 linear projection과 cross entropy를 fusion하고, linear 연산 결과는 chunking하여 backward path때 재계산하는 방식으로 activation 메모리를 줄였습니다. Qwen3-30B-A3B 의 hidden size, vocab size 기준으로 PyTorch baseline 대비 1.68배, Triton baseline 대비 1.02배의 speedup과 activation memory 72% 감소를 확인했습니다.

2. Analysis

FORWARD



BACKWARD



기존 방식은 LM head 결과인 logits [tokens, vocab/tp]을 full tensor로 HBM에 저장한 뒤, cross entropy 계산을 할 때 다시 읽습니다. vocab size가 큰 모델에서는 이 logits tensor가 tokens × local_vocab × dtype_size만큼 rank별 activation memory를 차지합니다.

Fused path에서는 GEMM 결과를 vocab split 단위로 처리하면서 epilogue에서 online softmax를 위해 max와 accumulator를 갱신합니다. 각 split의 logits tile은 cross entropy 계산에 바로 사용되고, full logits tensor로 오래 materialize하지 않습니다. 따라서 forward에서는 token logprob와 _max, _accu 같은 작은 auxiliary tensor만 남기면 됩니다.

Backward에서는 대신 split별로 필요한 logits를 재계산하고, partial d_logits을 이용해 d_hidden과 d_weight를 계산합니다. 추가 계산을 일부 허용하는 대신 HBM에 큰 logits tensor를 저장하고 다시 읽는 비용을 줄이는 구조입니다.

3. Implementation

- Hopper GPU에 맞는 cooperative schedule로 wgmma + epilogue 방식 구현
- CUTLASS/CuTe 기반 linear projection + cross entropy path fusion
- Forward: vocab split별 online cross entropy로 _max, _accu, token logprob 계산
- Backward: split별 partial d_logits buffer 재사용 후 d_hidden, d_weight 계산
- PyTorch 대비 speed/memory benchmark test 추가

4. Results

Megatron-LM 프로젝트에 PR #3345으로 제출했습니다.

Qwen 30B-A3B 기준 benchmark shape tokens=4096, vocab=129280, hidden=7168에서 PyTorch baseline 대비 forward latency 25.51ms -> 11.74ms, backward latency 57.68ms -> 37.75ms를 확인했습니다. Peak memory는 forward 9576MB -> 1889MB, backward 11596MB -> 4016MB로 감소했습니다. Triton baseline 에서는 1.02배의 speedup과 activation memory 72% 감소를 확인했습니다.

3. LLM Pretraining System Optimization

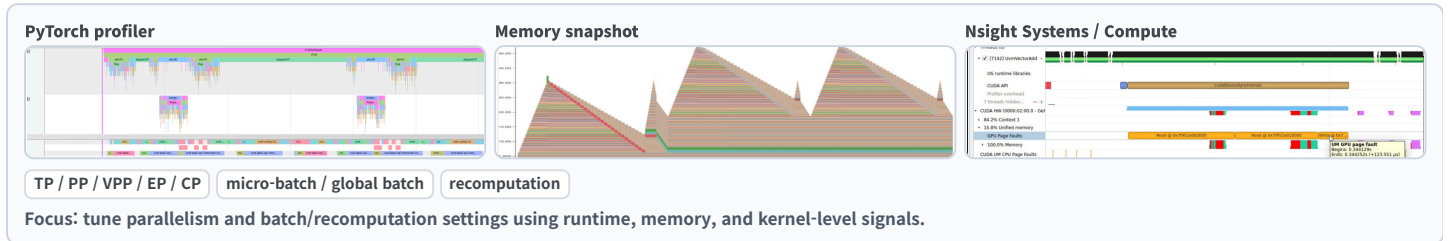
1. Summary

KT Foundation LLM 믿음 pretraining에서 Dense 및 100B급 MoE 모델을 대상으로 분산학습 설정을 최적화했습니다. 모델 구조와 클러스터 조건에 맞게 TP/PP/VPP/EP/CP 조합, batch/recomputation, 통신 overlap을 조정해 학습 처리량과 안정성을 함께 관리했습니다.

또한 BPB evaluation과 native benchmark evaluation을 구현하여, pretraining 중 품질 feedback을 더 빠르게 받을 수 있도록 했습니다. BPB evaluation은 training loop내에 삽입하여 빠르게 품질을 확인하는 용도로 사용하고, native benchmark evaluation은 Megatron-LM checkpoint를 직접 평가하도록 구현해 checkpoint 변환 오버헤드와 평가 지연을 줄였습니다.

2. Analysis

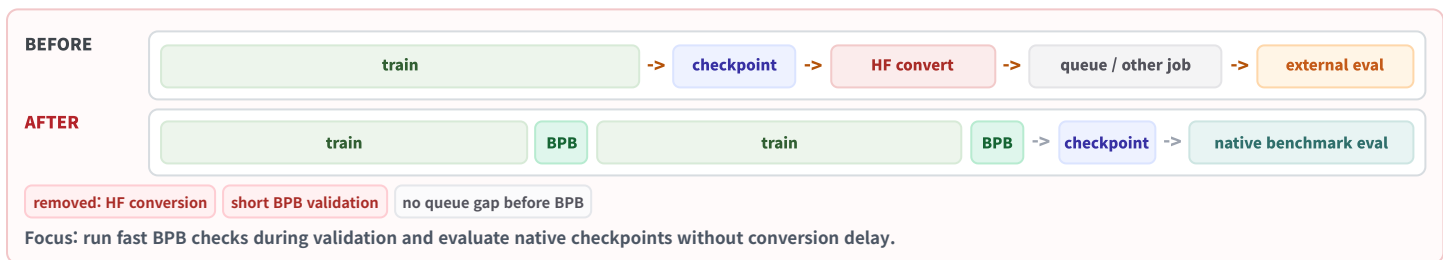
Distributed training optimization



대규모 pretraining에서는 parallelism과 batch 설정이 서로 다른 trade-off를 만듭니다. TP는 activation memory 부담을 줄일 수 있지만 통신 비용과 작은 kernel 실행을 늘릴 수 있고, PP는 모델을 나눠 담을 수 있지만 micro-batch와 gradient accumulation 수가 맞지 않으면 pipeline bubble이 커집니다. VPP, EP, CP도 각각 memory 절감, 통신 비용, overlap 가능성이 달라 한 가지 설정을 고정적으로 적용하기 어렵습니다.

그래서 profiler와 Nsight로 step 단위 유휴 시간, NCCL 통신 대역폭, 연산-통신 overlap을 확인하며 조합을 선택했습니다. memory snapshot은 activation memory 사용량이 batch/recomputation 설정을 감당할 수 있는지 확인하는 기준으로 사용했습니다.

Evaluation pipeline optimization



기존 evaluation pipeline은 checkpoint 저장 이후 평가까지의 과정에 실패 지점과 운영 비용이 많았습니다. HF format 변환과 별도 evaluation job이 필요해 변환 job이 GPU 노드를 점유했고, queue 지연이나 변환 오류가 생기면 평가가 안정적으로 이어지지 않았습니다. 또한 학습에 사용한 Megatron-LM kernel이 아니라 변환된 HF checkpoint 기준의 다른 evaluation kernel로 평가하게 되어, 문제가 생겼을 때 원인이 학습 품질인지 평가 파이프라인인지 분리해 확인해야 했습니다.

이를 개선하기 위해 Megatron-LM checkpoint를 그대로 평가하는 native benchmark evaluation을 구현해 변환 단계를 제거했습니다. BPB evaluation은 validation 단계에 포함해, 오래 걸리는 benchmark 평가 없이도 학습 상태를 빠르게 확인할 수 있게 했습니다. 이를 통해 평가 과정에서 막히는 지점을 줄이고, 결과를 다음 training recipe에 더 안정적으로 반영할 수 있었습니다.

3. Implementation

- Dense / 100B급 MoE 구조에 맞는 TP/PP/VPP/EP/CP 조합 검토
- micro-batch, global batch, gradient accumulation, recomputation 설정 조정
- profiler와 Nsight로 유휴 시간, 통신 대역폭, 연산-통신 overlap 확인
- BPB evaluation을 train loop 내부 validation 단계에 통합
- Megatron-LM checkpoint native benchmark evaluation 구현 및 HF conversion 제거

4. Results

분산학습 설정 최적화와 evaluation pipeline 개선은 KT Foundation LLM 믿음 pretraining에 사용했습니다. 분산학습 설정은 모델 구조와 클러스터 조건에 맞춰 TP/PP/VPP/EP/CP, batch/recomputation, 통신 overlap 조합을 선택하는 기준으로 활용했고, 학습 처리량과 안정성을 함께 관리했습니다.

Evaluation pipeline에서는 BPB evaluation과 native benchmark evaluation을 도입해 checkpoint 이후 평가 과정의 변환 단계와 실패 지점을 줄였습니다. 이를 통해 학습 상태를 더 안정적으로 확인하고, 평가 결과를 다음 training recipe에 반영하는 흐름을 만들었습니다.